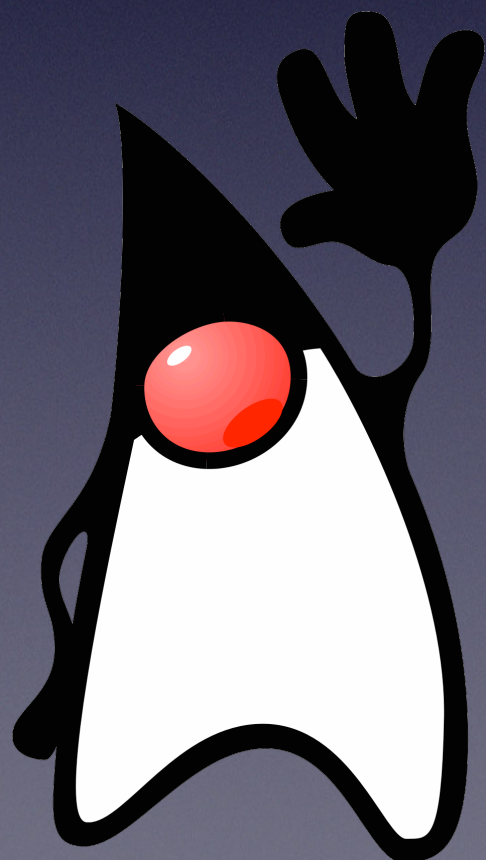


Programación

8 h./semana



Java™



Duke

Programación

Unidad I. Introducción



I. Sumar, restar, multiplicar o dividir 2 números



1. Sumar, restar, multiplicar o dividir 2 números
2. Comparar 2 valores



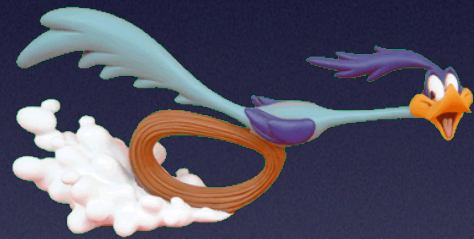
1. Sumar, restar, multiplicar o dividir 2 números
2. Comparar 2 valores
3. Almacenar y recuperar información



Operaciones sencillas ... pero ...



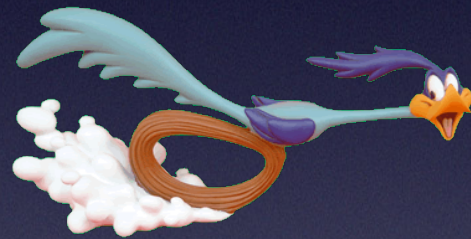
Operaciones sencillas ... pero ...



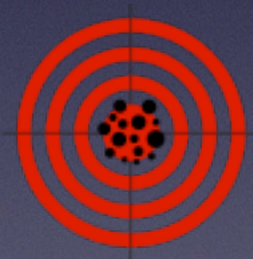
muy rápido



Operaciones sencillas ... pero ...



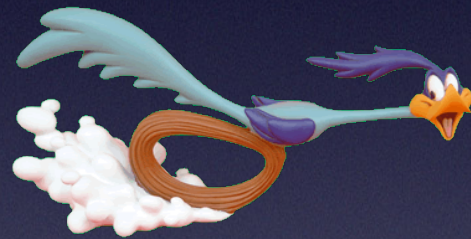
muy rápido



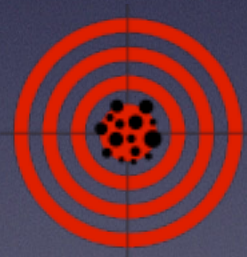
fiable



Operaciones sencillas ... pero ...



muy rápido



fiable



gran capacidad

Algoritmo / Programa

Algoritmo

Algoritmo: es un **procedimiento** paso a paso para resolver un problema en una **cantidad de tiempo finita**.



Algoritmo / Programa

Algoritmo

Algoritmo: es un **procedimiento** paso a paso para resolver un problema en una **cantidad de tiempo finita**.



Procedimiento: es una **secuencia** de instrucciones que pueden ejecutarse de manera **mecánica**.

Algoritmo / Programa

Algoritmo

Juego de las 15 monedas

Este juego para dos jugadores comienza colocando 15 monedas sobre la mesa. El jugador uno elige de 1 a 3 monedas. A continuación, el jugador dos elige de 1 a 3 monedas de las restantes sobre la mesa. Este proceso se repite hasta que no queden monedas. El jugador que se vea obligado a tomar la última moneda pierde.

El problema consiste en encontrar una estrategia ganadora que permita al jugador 1 obligar al jugador 2 a tomar la última moneda.

Algoritmo / Programa

Algoritmo

Juego de las 15 monedas

Este juego para dos jugadores comienza colocando 15 monedas sobre la mesa. El jugador uno elige de 1 a 3 monedas. A continuación, el jugador dos elige de 1 a 3 monedas de las restantes sobre la mesa. Este proceso se repite hasta que no queden monedas. El jugador que se vea obligado a tomar la última moneda pierde.

El problema consiste en encontrar una estrategia ganadora que permita al jugador 1 obligar al jugador 2 a tomar la última moneda.

- 1.- Jugador 1 coge 2 monedas
- 2.- Jugador 2 coge x monedas ($x \geq 1$ y $x \leq 3$)
- 3.- Jugador 1 coge $4-x$ monedas
- 4.- Repetir pasos 2 y 3 hasta que solo queda 1 moneda sobre la mesa.

Lenguaje de programación: es un conjunto de reglas, símbolos y palabras especiales (reservadas) usadas para construir un programa.

```
Void prueba() {  
    i:= 0;  
    while (i < 9) do {  
        printf("%d\n",i);  
        i++;  
    }  
    ...  
}
```



```
11100011000010000  
10011111010101010  
100001111010101  
01011100011010101  
1111111101110101  
01111101010101010  
101010  
...  
.
```



Lenguaje de programación: es un conjunto de reglas, símbolos y palabras especiales (reservadas) usadas para construir un programa.

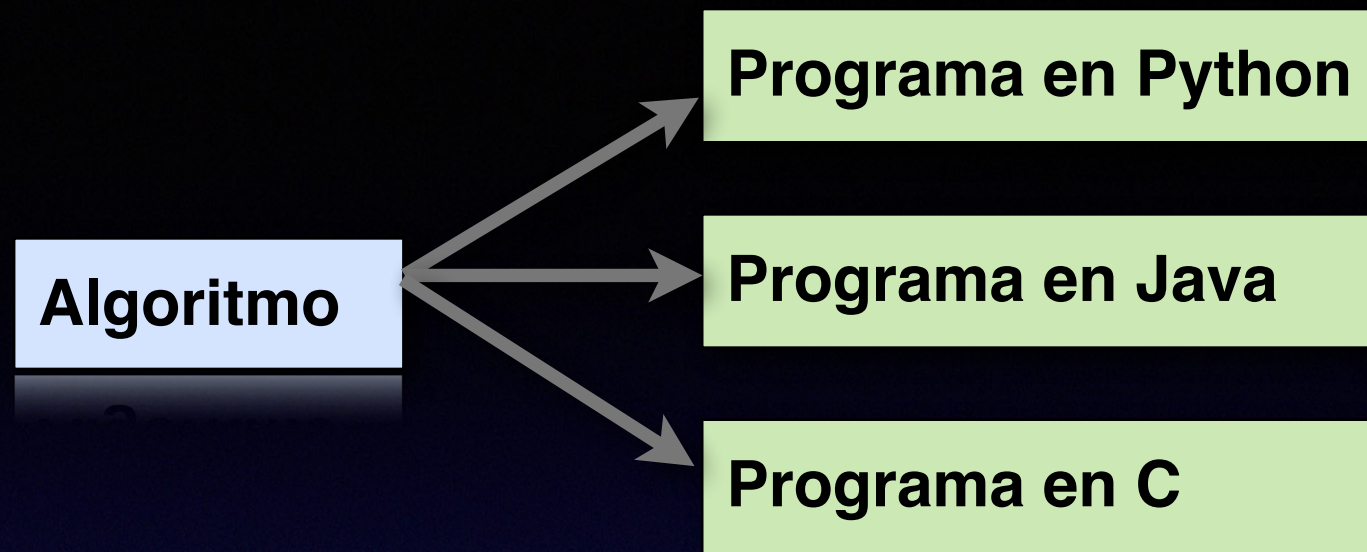
Programa: Es una secuencia de instrucciones que indican acciones que se van a ejecutar por un ordenador.

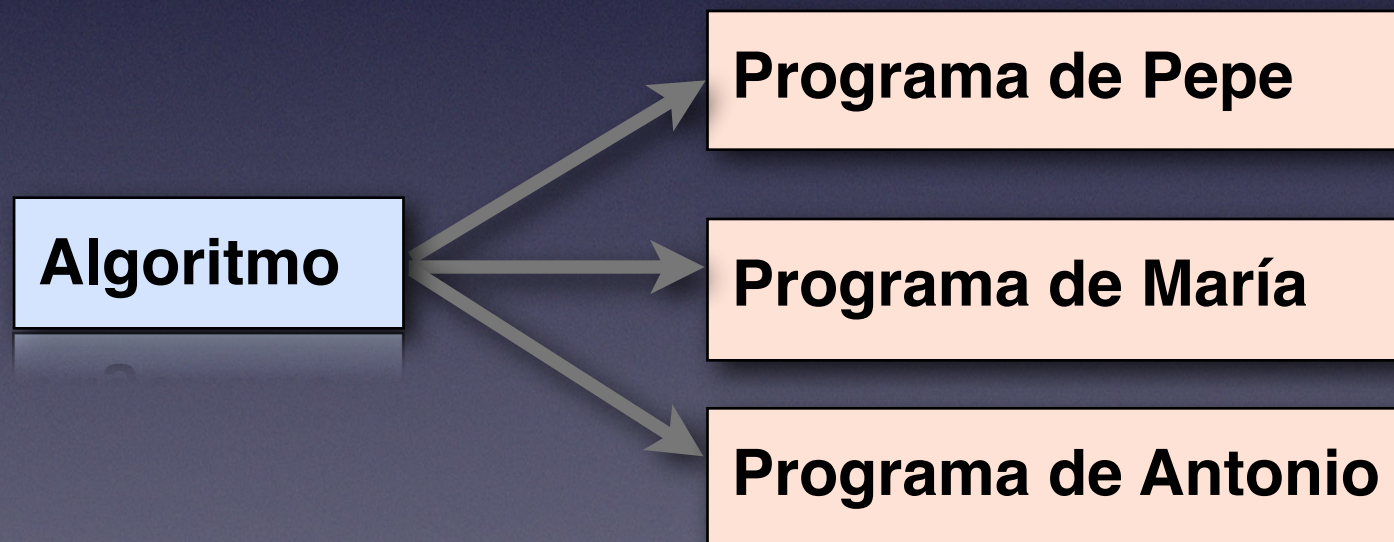
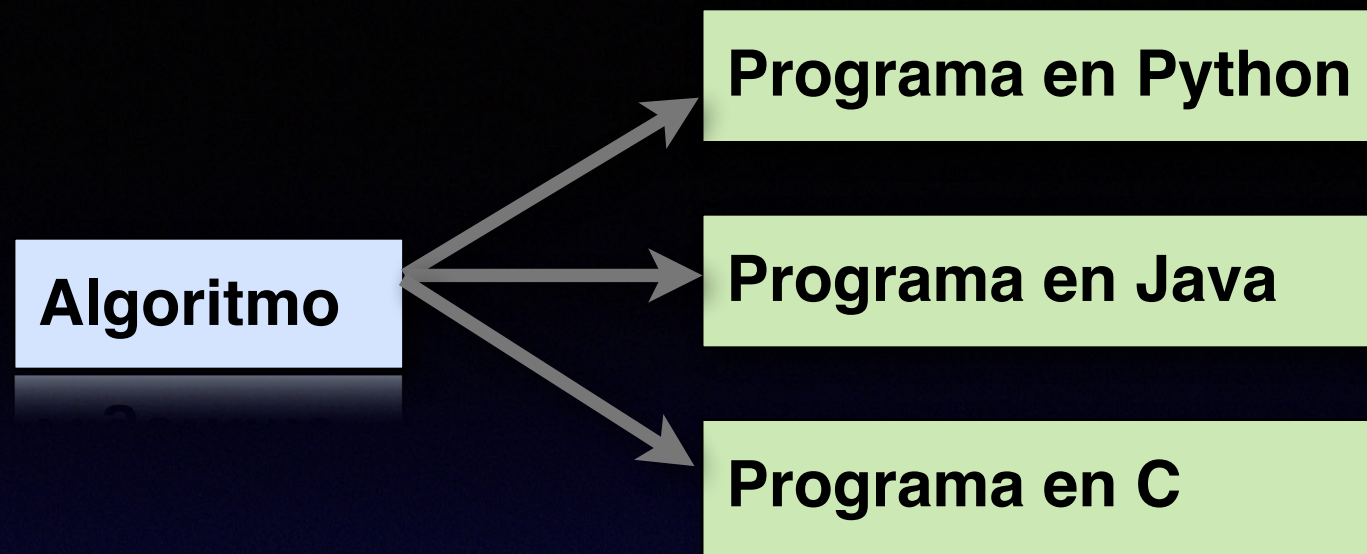
```
Void prueba() {  
    i:= 0;  
    while (i < 9) do {  
        printf("%d\n",i);  
        i++;  
    }  
    ...  
}
```

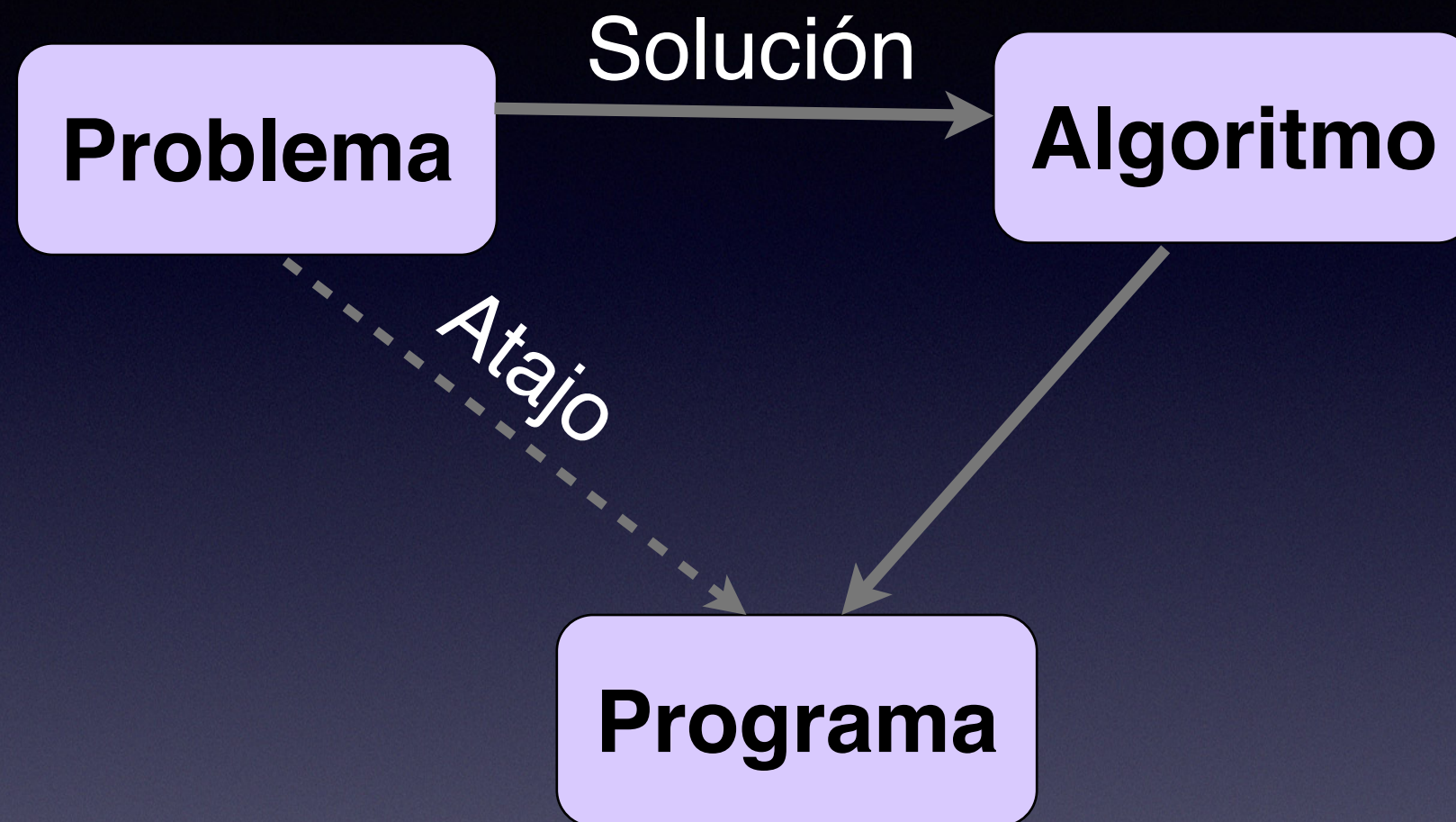


```
11100011000010000  
10011111010101010  
100001111010101  
01011100011010101  
1111111101110101  
01111101010101010  
101010  
...
```



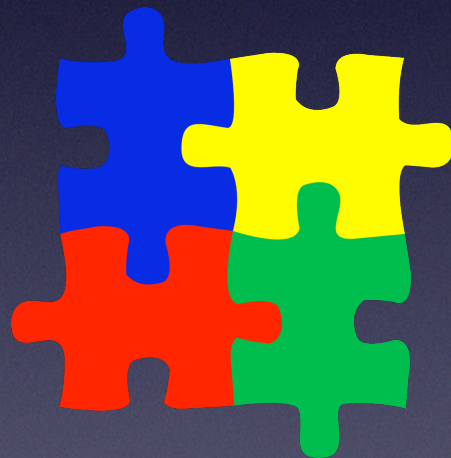






Ciclo de vida de una aplicación:

Una **aplicación** se compone de uno o varios programas interrelacionados para llevar a cabo una cierta tarea de forma automática usando un ordenador.



Ciclo de vida de una aplicación:



Ciclo de vida de una aplicación: Es el proceso que se sigue desde la identificación del problema o tarea hasta que tenemos una solución instalada y funcionando en los ordenadores de los usuarios, siempre y cuando la Aplicación sea útil para ellos.

Ciclo de vida de una aplicación:



Fases de diseño

Fases de instalación

Fases de diseño:

Problema

Fases de diseño:

Problema

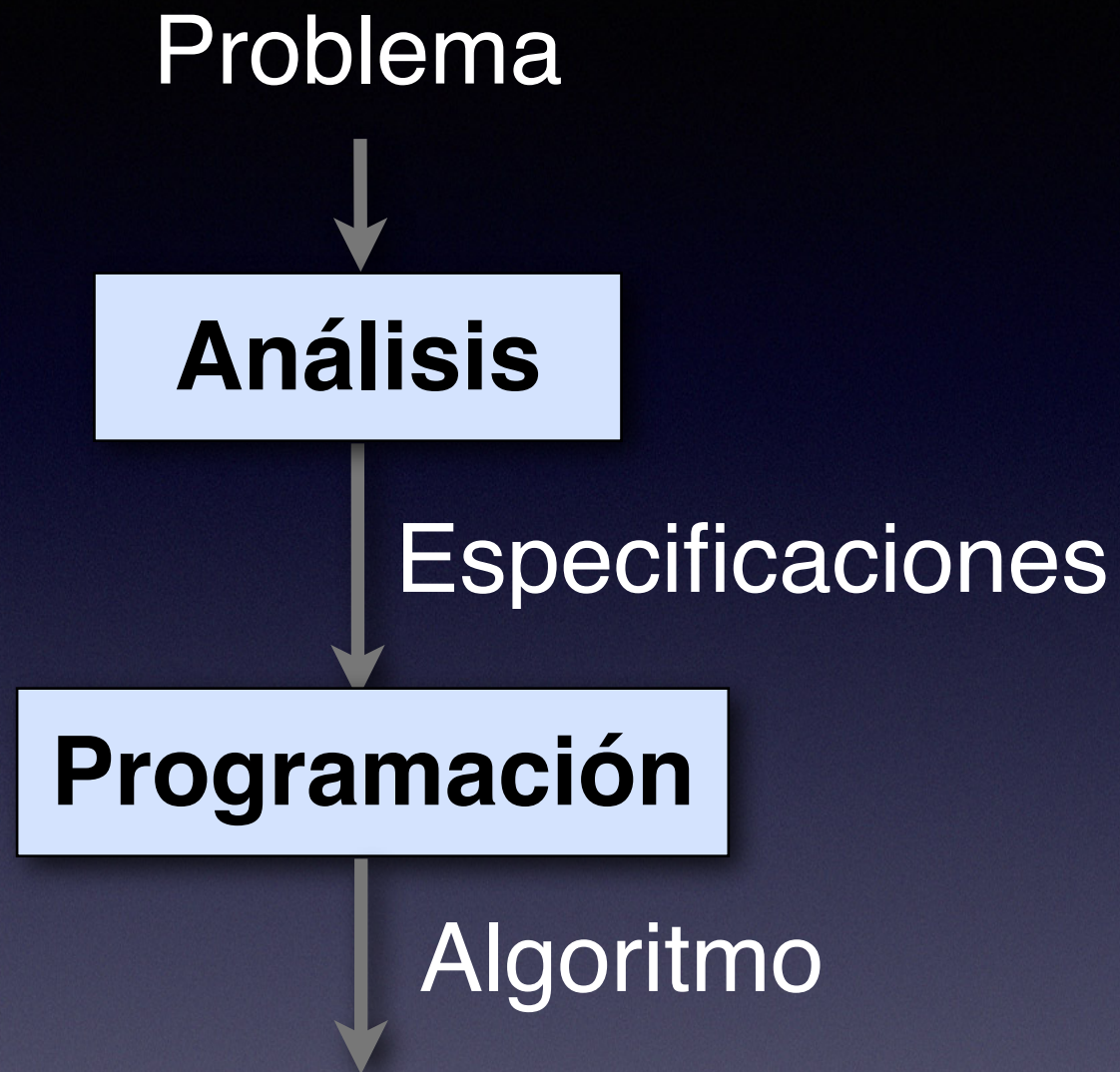


Análisis

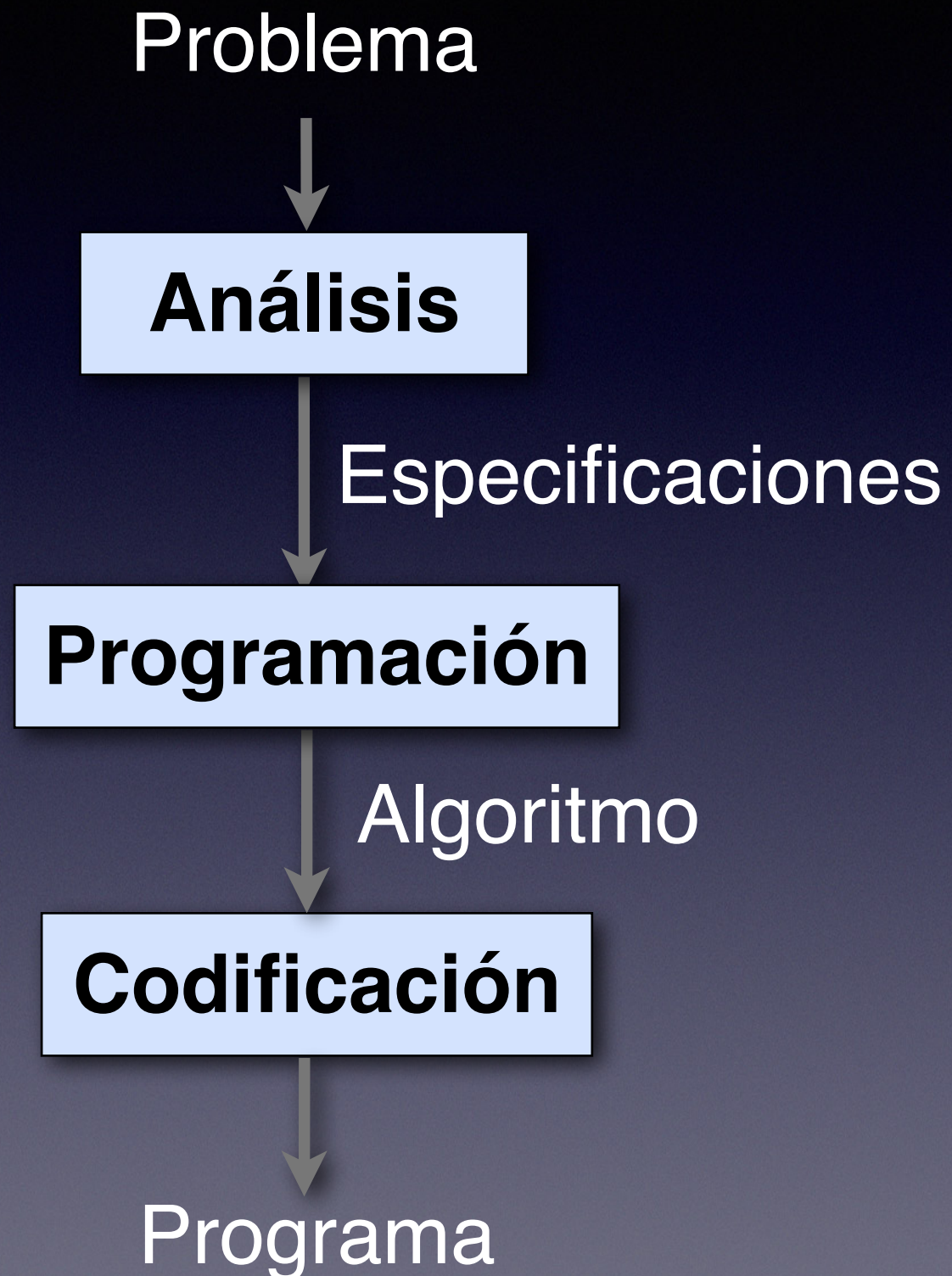


Especificaciones

Fases de diseño:

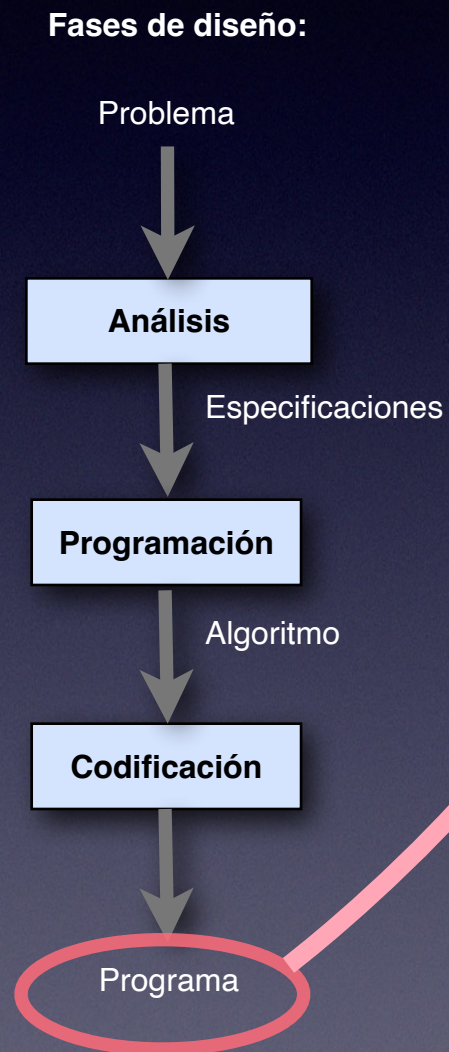


Fases de diseño:

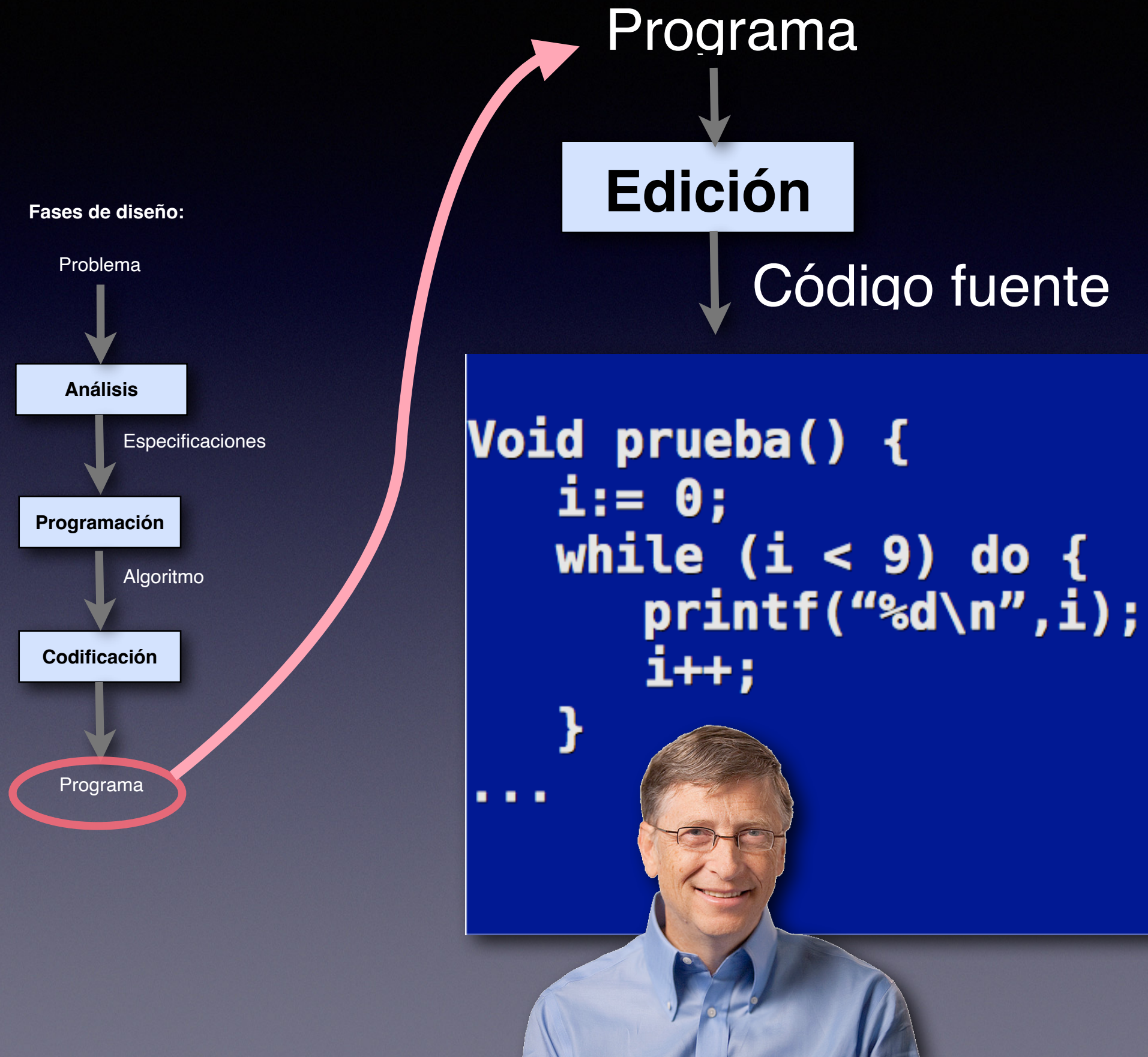


Fases de instalación

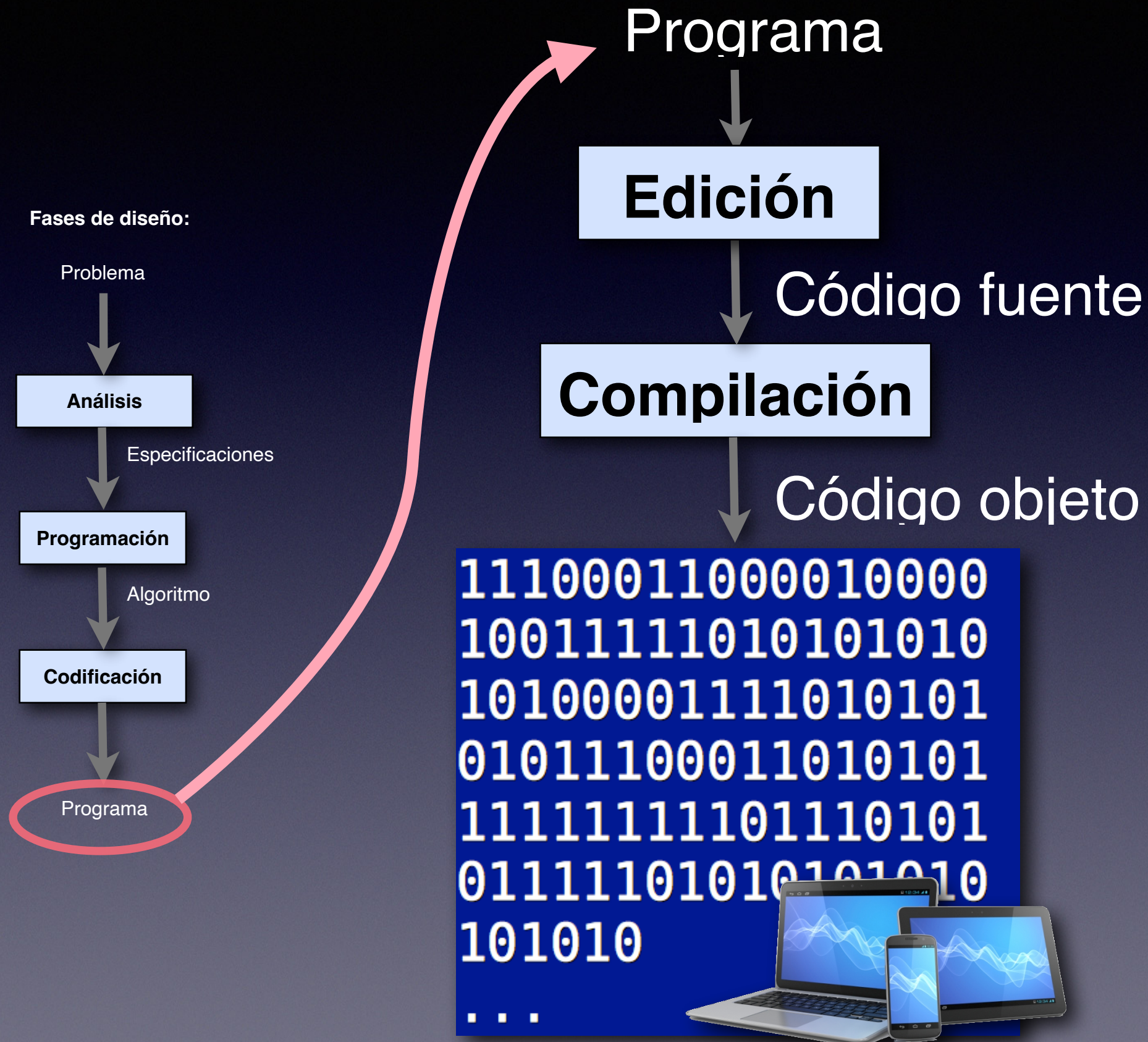
Programa



Fases de instalación



Fases de instalación



Un compilador: traduce **todo** el código fuente del programa en código objeto.

Un Intérprete: traduce el código fuente **línea a línea** a medida que se va ejecutando el programa

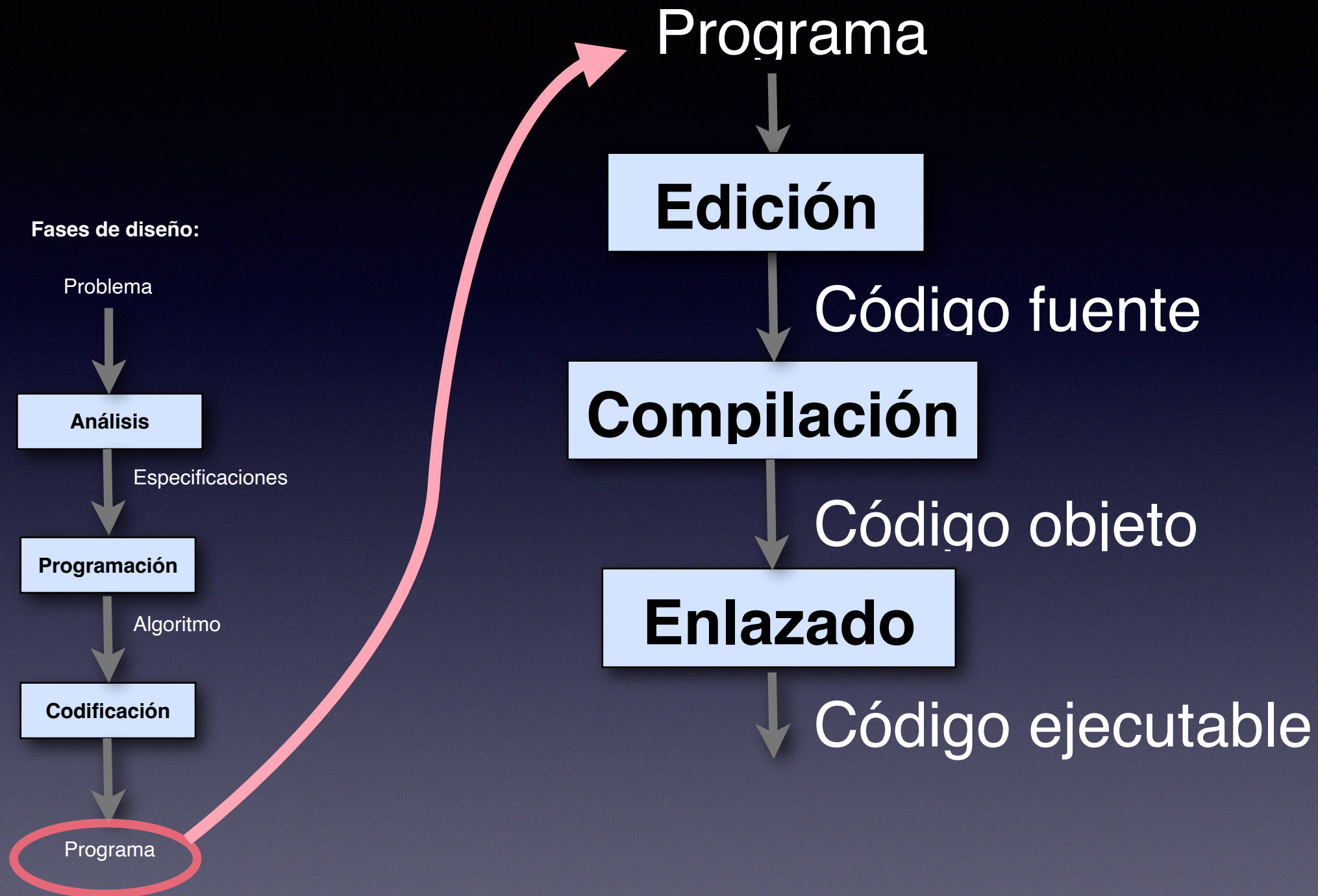
```
Void prueba() {  
    i:= 0;  
    while (i < 9) do {  
        printf("%d\n",i);  
        i++;  
    }  
    ...  
}
```



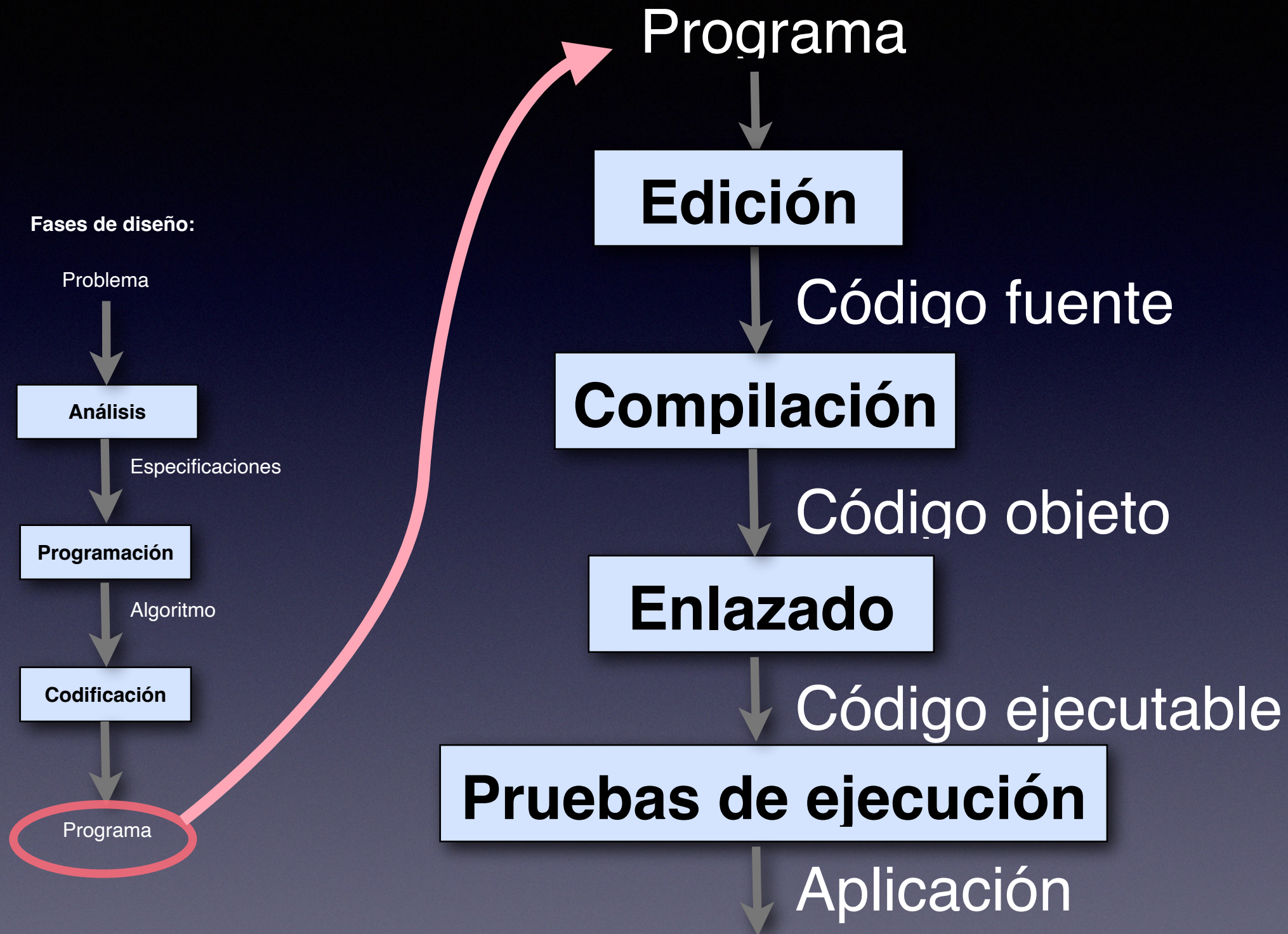
```
11100011000010000  
100111110101010  
100001111010101  
010111000110101  
1111111101110101  
011111010101010  
101010  
...
```



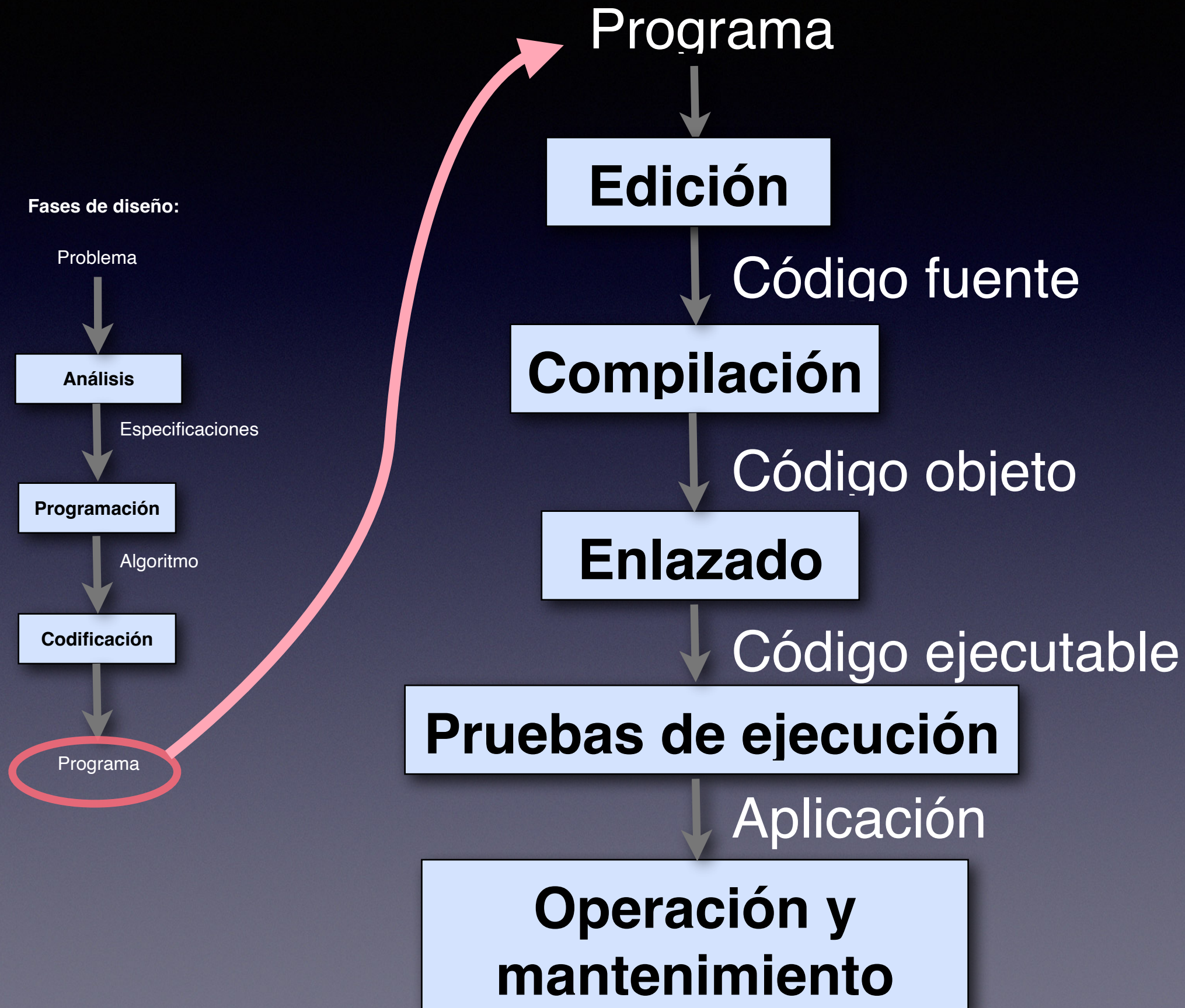
Fases de instalación:



Fases de instalación:



Fases de instalación:



Fases de diseño: Fases de instalación:



Fases de diseño: Fases de instalación:



Fases de diseño: Fases de instalación:



Fases de diseño:

Fases de instalación:



Fases de diseño: Fases de instalación:



Calidad de los programas

Calidad de los programas



Legibilidad

Calidad de los programas



Legibilidad



Portabilidad

Calidad de los programas



Legibilidad



Portabilidad



(Fácil de escribir en diferentes lenguajes)

Calidad de los programas



Legibilidad



Portabilidad



Modificabilidad

Calidad de los programas



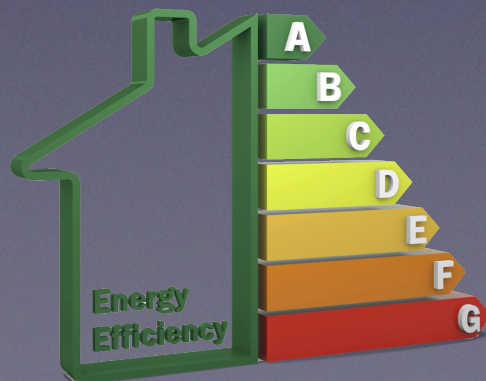
Legibilidad



Portabilidad



Modificabilidad



Eficiencia

Calidad de los programas



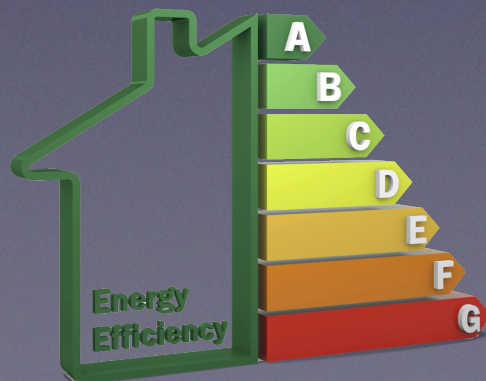
Legibilidad



Portabilidad

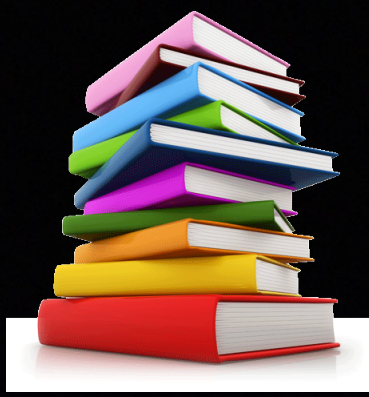


Modificabilidad



Eficiencia

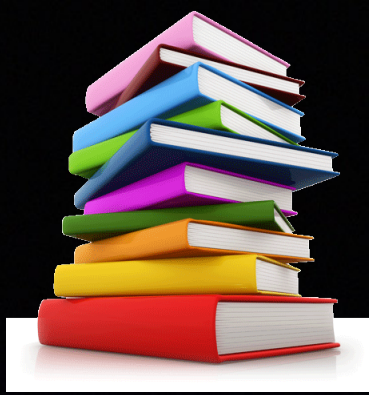
Metodología de programación es el conjunto de métodos y técnicas que ayudan a desarrollar programas que siguen todos los requisitos de calidad expuestos anteriormente.



Documentación del Programa

Documentación interna

```
Void prueba() {  
    i:= 0;  
    while (i < 9) do {  
        printf("%d\n",i);  
        i++;  
    }  
    ...  
}
```

Documentación del Programa

Documentación interna

```
Void prueba() {  
    i:= 0;  
    while (i < 9) do {  
        printf("%d\n",i);  
        i++;  
    }  
    ...  
}
```

Comentarios:

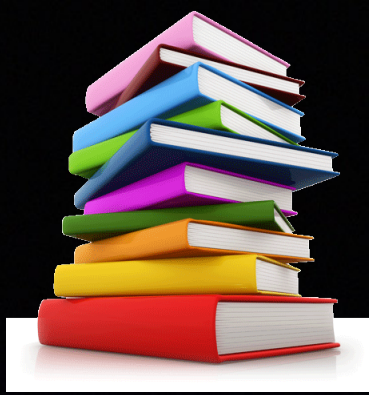
```
// This is comment  
/* This is a comment with  
two or more lines */
```

Código autocomentado:

Identificadores adecuados y con significado

Declaración de constantes para valores fijos

Indentación, líneas vacías, etc.



Documentación del Programa

Documentación interna

```
Void prueba() {  
    i:= 0;  
    while (i < 9) do {  
        printf("%d\n",i);  
        i++;  
    }  
    ...  
}
```

Documentación externa



- Especificaciones del análisis.
- Descripción y diagramas del diseño de los programas
- Manual de usuario
- Manual de mantenimiento